

# What is DSPy



DSPy is a GenAI authoring framework, that simplifies the development of GenAI applications.

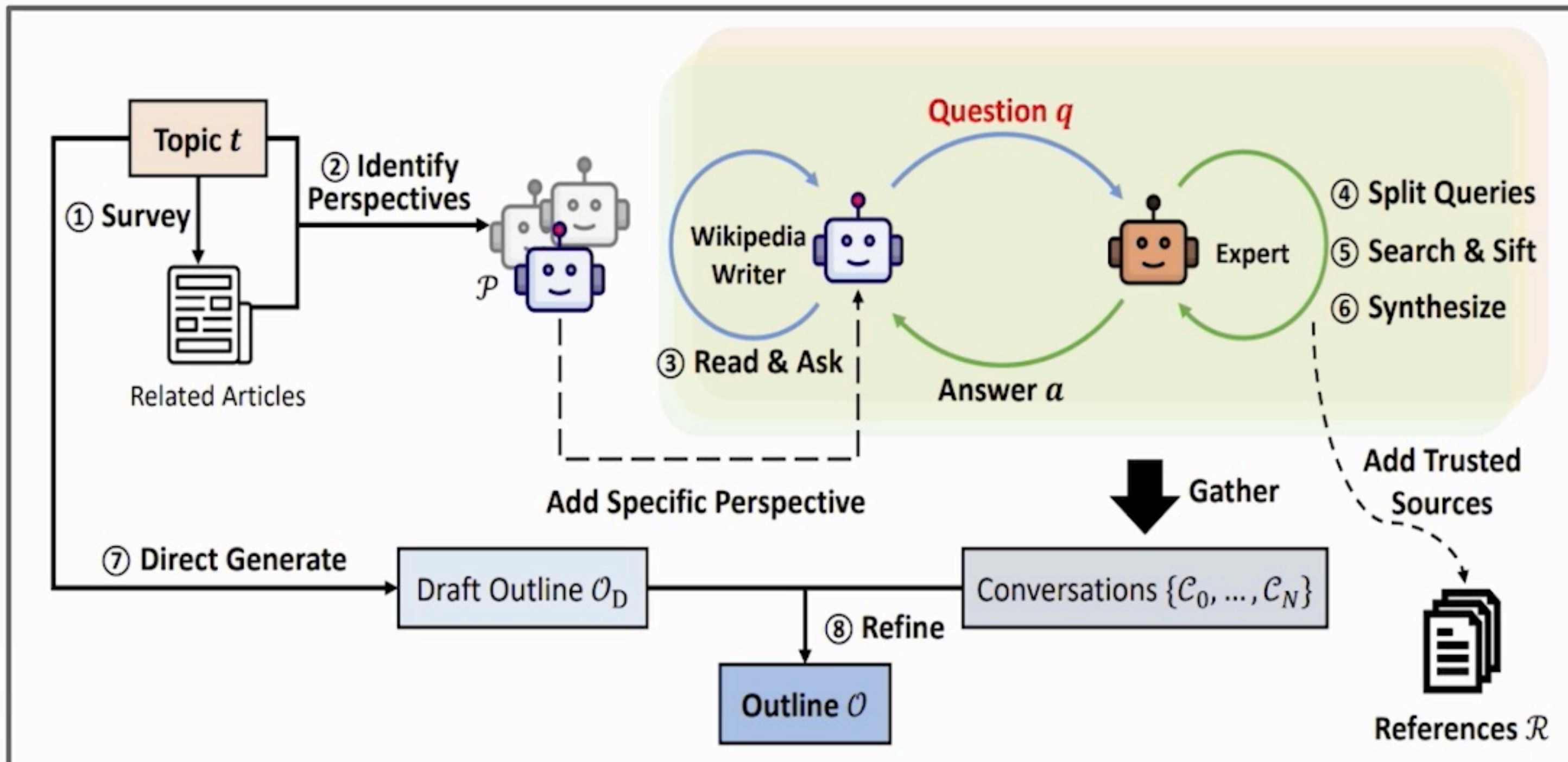
## Use Cases

- Retrieval-augmented generation (RAG)
- Chain-of-thought prompting
- Structured output generation (like JSON)
- LLM pipelines for agents or tools

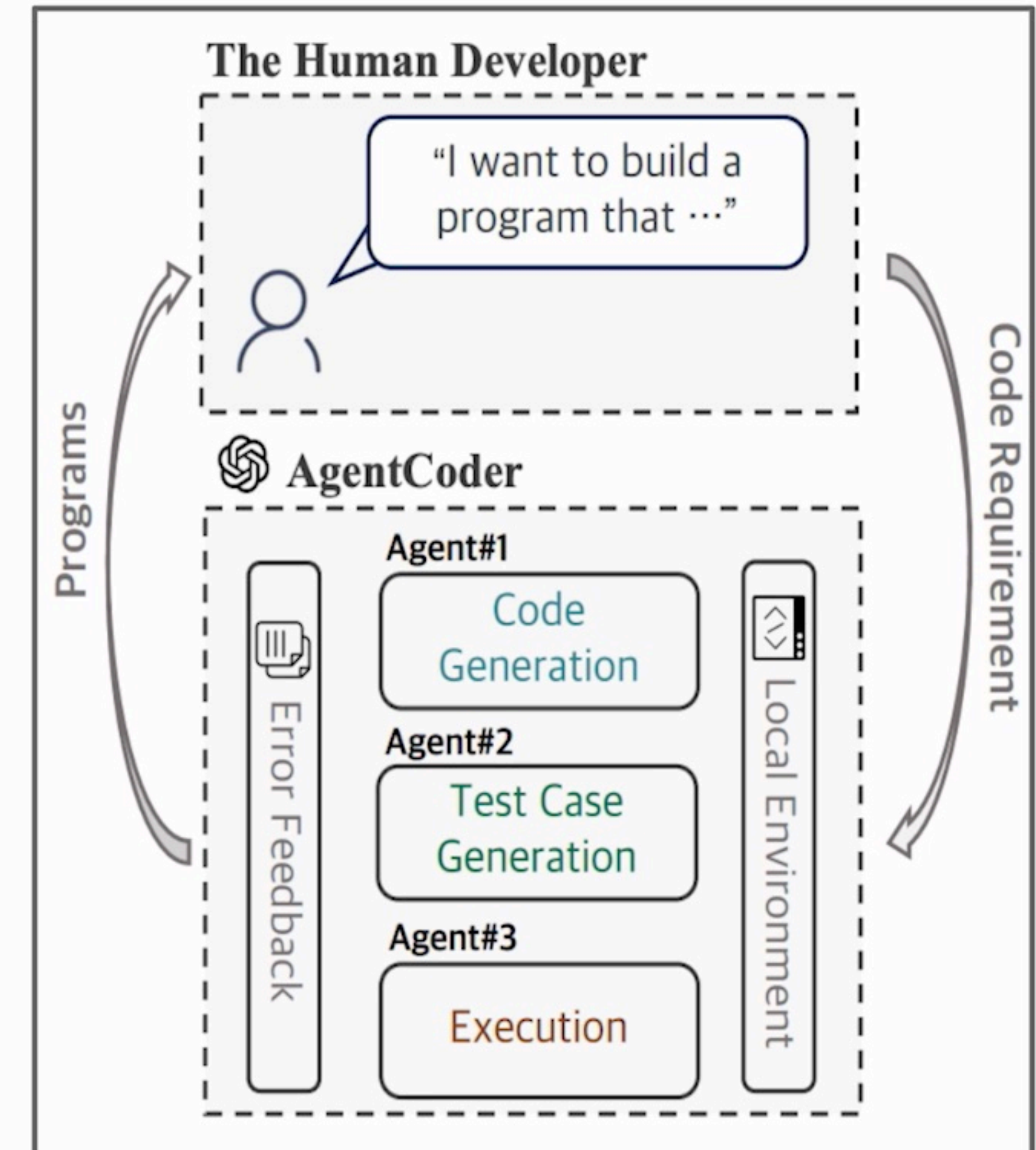


# The Rise of Compound AI System

Question Answering AI system



Code Generator AI system





# Prompt Engineering is Messy

- ✅ **Better prompts = better results** (But getting there isn't straightforward...)

**✗ Prompt engineering is messy**

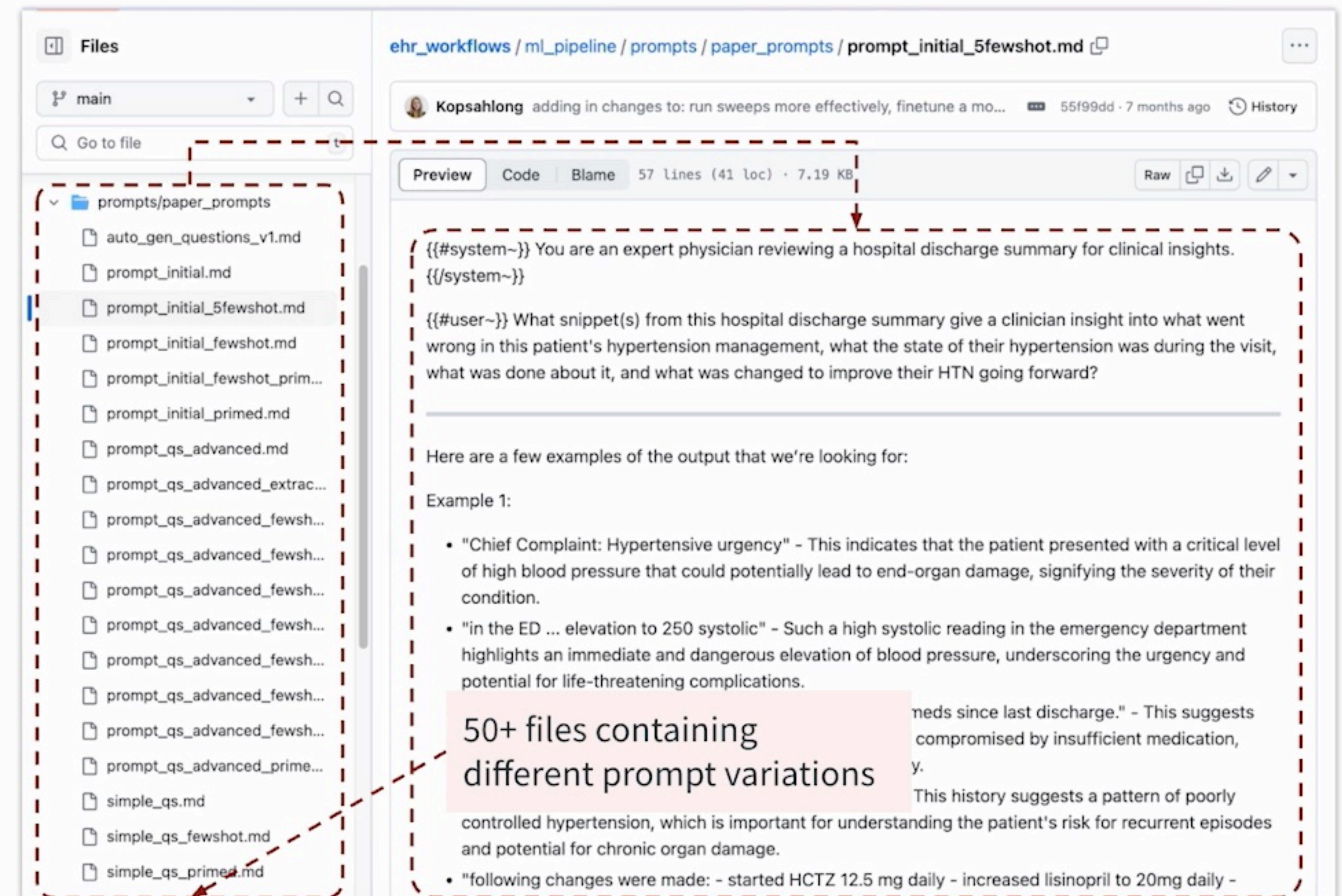
- Involves trial-and-error with string tweaks
- Unclear what changes actually help

 **High iteration cost**

- 50–100+ prompt variants per task
- Prompts can span **thousands of words**

 **Model bias makes it worse**

- Switching models = redoing all prompt tuning





# Framework Can be Additional Troublemaker

## ⚙️ Frameworks simplify complex tasks

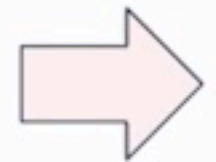
- Great for building agents, RAG pipelines, etc.

## 😞 Users often feel frustrated

- “The framework adds more pain than value.”

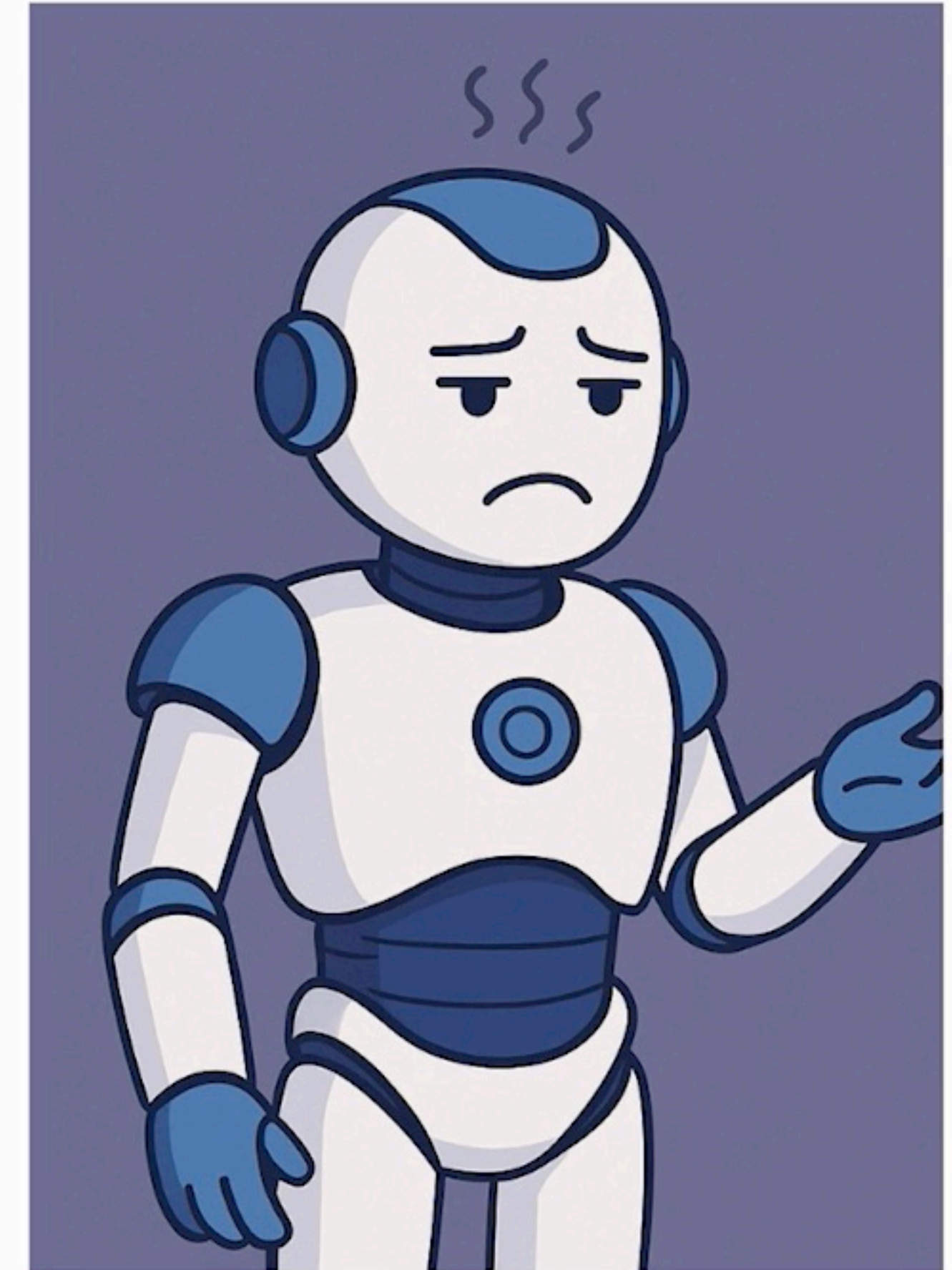
## 📖 Framework contract overhead

- Requires learning custom APIs and design patterns



## 🔒 Lock-in and migration pain

- Existing code becomes tightly coupled; Hard to switch frameworks later





This is the problem that DSPy aims solve.



# DSPy

*Programming* — not prompting — Language Models

## DSPy is a lightweight, flexible framework

- Simplifies interaction with language models (LMs)

## Automatic program optimization

- Prompt optimization
- Fine-tuning LM weights with **DSPy Optimizer**

## Ready for production

- Supports **streaming**, **async**, and more out of the box

## Modular building blocks, no lock-in

- Use only what you need
- Easy to migrate **to** or **from** DSPy

# What is Special About DSPy?

**LM-agnostic programing,  
not LM-biased prompting**

Lesson 2

**Seamless Productization,  
native features and MLflow  
integration**

Lesson 3

**Automatic Program  
Optimization**

Lesson 4